Labprogram: 8

a)Add animations to UI elements using flutter's animation framework

```
import 'package:flutter/material.dart'; void main() {
runApp(MyApp());
}

class MyApp extends StatelessWidget { @override
Widget build(BuildContext context) { return MaterialApp(
home: Scaffold( appBar: AppBar(
title: Text('Animation Example'),
),
body: AnimationWidget(),
),
);
}
}

class AnimationWidget extends StatefulWidget { @override
_AnimationWidgetState createState() => _AnimationWidgetState();
}

class _AnimationWidgetState extends State<AnimationWidget> with
SingleTickerProviderStateMixin {
late AnimationController _controller; late Animation<double> _animation;

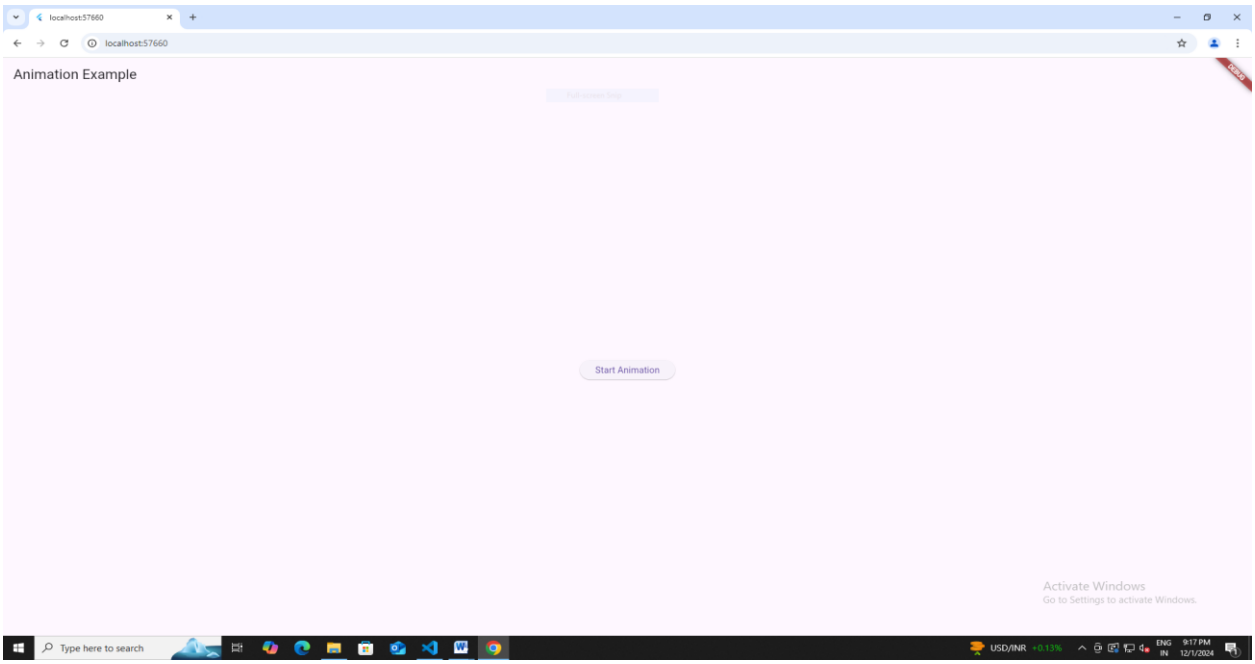
@override
void initState() { super.initState();
_controller = AnimationController( duration: Duration(seconds: 1), vsync: this,
);
_animation = Tween<double>(begin: 0, end: 300).animate(_controller)
..addListener(() {
setState(() {}); // Trigger rebuild when animation value changes
});
}

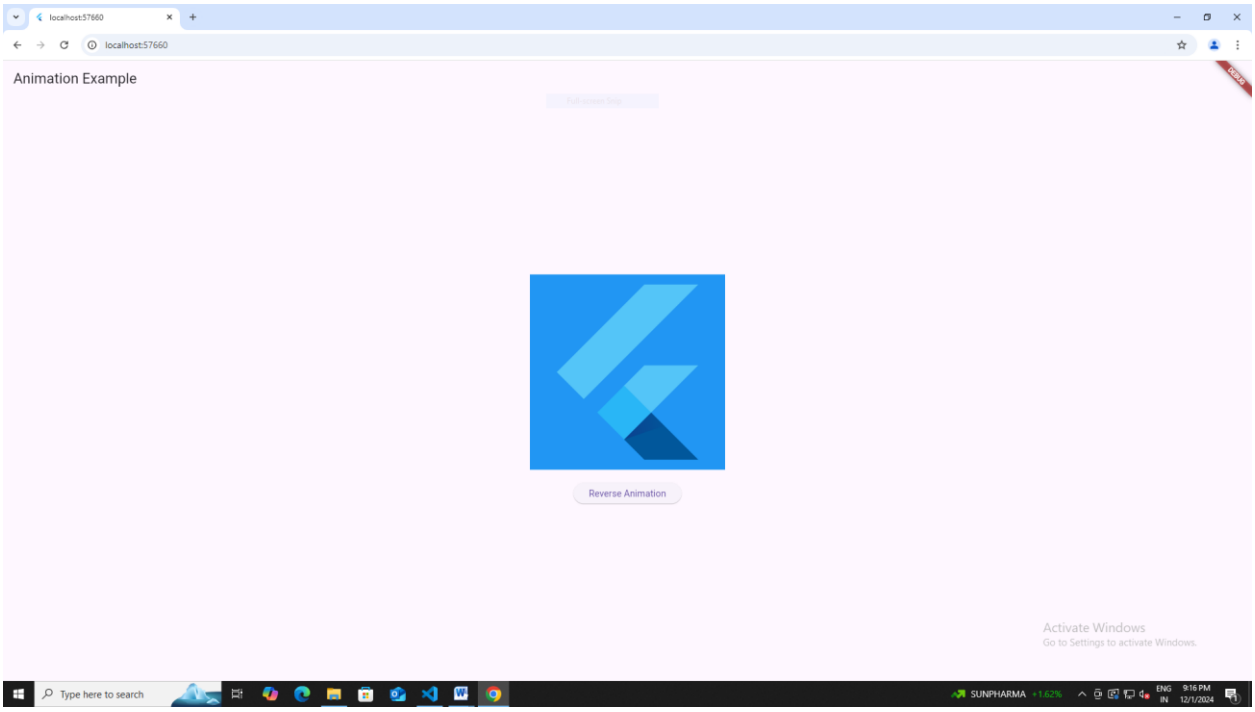
@override
Widget build(BuildContext context) { return Center(
child: Column(
mainAxisAlignment: MainAxisAlignment.center,
children: <Widget>[
Container(
width: _animation.value, height: _animation.value, color: Colors.blue,
child: FlutterLogo(size: 100),
```

```
),
SizedBox(height: 20), ElevatedButton( onPressed: () {
if (_controller.status == AnimationStatus.completed) {
_controller.reverse();
} else {
_controller.forward();
}
},
child: Text(
_controller.status == AnimationStatus.completed
? 'Reverse Animation'
: 'Start Animation',
),
),
],
),
);
}

@override
void dispose() {
_controller.dispose(); super.dispose();
}
}
```

Output:





We define an Animation object with Tween to define the range of values for the animation.I nside the initState() method, we initialize the animation controller and define the animation.We use addListener() to trigger a rebuild when the animation value changes.

In the build method, we use the animated value _animation.value to control the size of the Container, whichcontains the FlutterLogo.

The ElevatedButton toggles the animation between forward and reverse based on the status of the animationcontroller.
You can customize the animation further by adjusting the duration, adding curves, or chaining multipleanimations together.

b)Experiment with different types of animations like fade,slide,etc.

Fade Animation:

```
import 'package:flutter/material.dart';

void main() { runApp(MyApp());
}

class MyApp extends StatelessWidget { @override
Widget build(BuildContext context) { return MaterialApp(
home: Scaffold( appBar: AppBar(
title: Text('Fade Animation Example'),
),
body: FadeAnimation(),
),
);
}
}
class FadeAnimation extends StatefulWidget {
@override
// ignore: library_private_types_in_public_api
_FadeAnimationState createState() => _FadeAnimationState();
}

class _FadeAnimationState extends State<FadeAnimation> with
SingleTickerProviderStateMixin { late AnimationController _controller;
late Animation<double> _animation;

@override
void initState() { super.initState();
_controller = AnimationController( vsync: this,
duration: Duration(seconds: 2),
);
_animation = Tween<double>( begin: 0.0,
end: 1.0,
).animate(_controller);

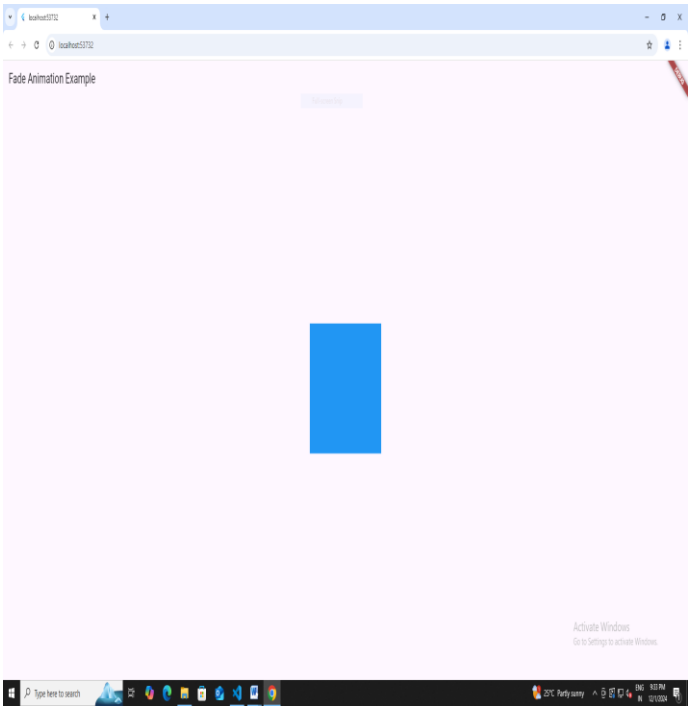
_controller.forward();
}


@override
Widget build(BuildContext context) { return Center(
child: FadeTransition( opacity: _animation, child: Container( width: 200,
height: 200,
```

```
color: Colors.blue,
),
),
);
}

@override
void dispose() {
_controller.dispose(); super.dispose();
}
}
```

Output:



Slide Animation:

```
import 'package:flutter/material.dart';

void main() { runApp(MyApp());
}

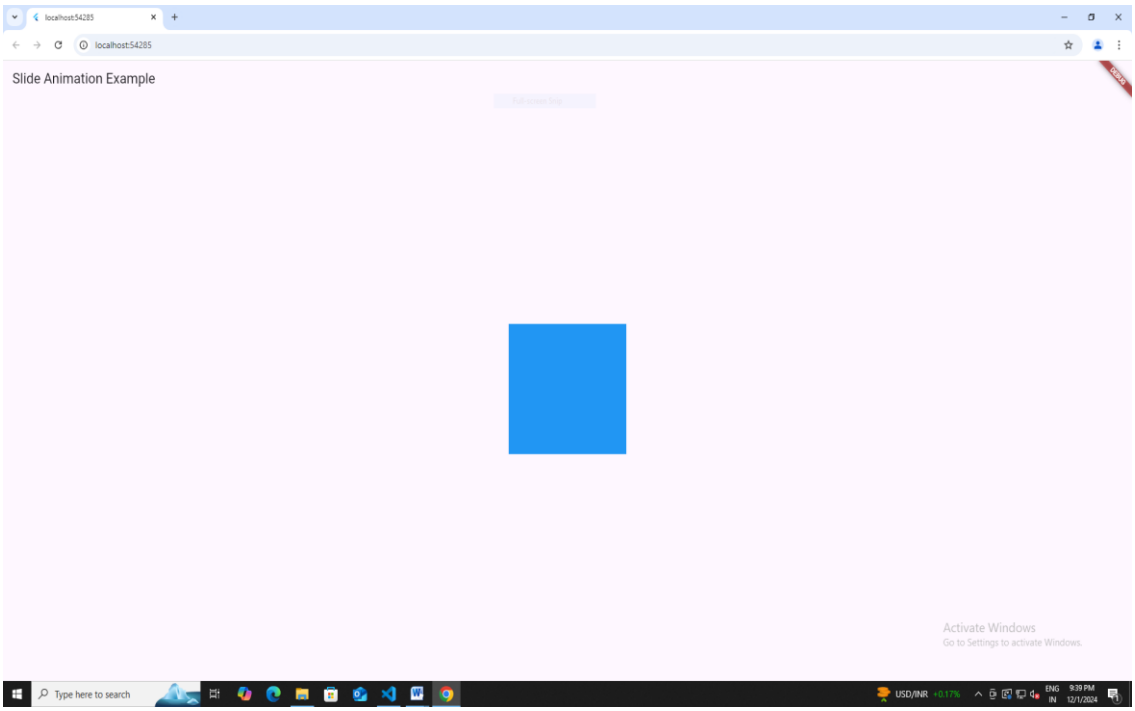
class MyApp extends StatelessWidget { @override
Widget build(BuildContext context) { return MaterialApp(
home: Scaffold( appBar: AppBar(
title: Text('Slide Animation Example'),
),
body: SlideAnimation(),
),
);
}
}

class SlideAnimation extends StatefulWidget { @override
// ignore: library_private_types_in_public_api
_SlideAnimationState createState() => _SlideAnimationState();
}
class _SlideAnimationState extends State<SlideAnimation> with
SingleTickerProviderStateMixin { late AnimationController _controller;
late Animation<Offset> _animation; @override
void initState() { super.initState();
_controller = AnimationController( vsync: this,
duration: Duration(seconds: 2),
);
_animation = Tween<Offset>( begin: Offset(-1.0, 0.0),
end: Offset(0.0, 0.0),
).animate(_controller);
_controller.forward();
}

@override
Widget build(BuildContext context) { return Center(
child: SlideTransition( position: _animation, child: Container( width: 200,
height: 200,
color: Colors.blue,
),
),
);
}
```

```
@override
void dispose() {
_controller.dispose(); super.dispose();
}
}
```

Output:



Scale Animation:

```
import 'package:flutter/material.dart';

void main() { runApp(MyApp());
}

class MyApp extends StatelessWidget { @override
Widget build(BuildContext context) { return MaterialApp(
home: Scaffold( appBar: AppBar(
title: Text('Scale Animation Example'),
),
body: ScaleAnimation(),
),
);
```

```
}
}

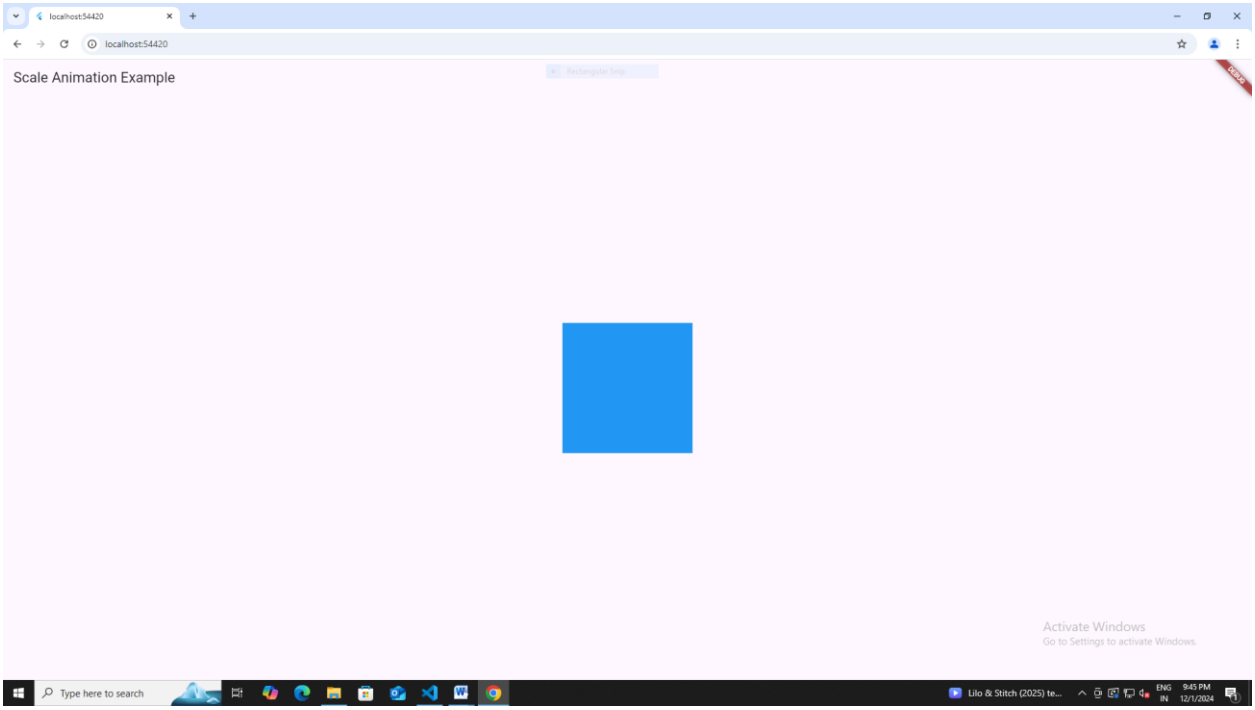
class ScaleAnimation extends StatefulWidget { @override
// ignore: library_private_types_in_public_api
_ScaleAnimationState createState() => _ScaleAnimationState();
}

class _ScaleAnimationState extends State<ScaleAnimation> with
SingleTickerProviderStateMixin { late AnimationController _controller;
late Animation<double> _animation;

@override
void initState() { super.initState();
_controller = AnimationController( vsync: this,
duration: Duration(seconds: 2),
);
_animation = Tween<double>( begin: 0.0,
end: 1.0,
).animate(_controller);
_controller.forward();
}

@override
Widget build(BuildContext context) { return Center(
child: ScaleTransition( scale: _animation, child: Container( width: 200,
height: 200,
color: Colors.blue,
),
),
);
}
@override
void dispose() {
_controller.dispose(); super.dispose();
}
}
```

Output:


localhost:54420
localhost:54420
Scale Animation Example
Activate Windows
Go to Settings to activate Windows.
Type here to search
Lilo & Stitch (2025) te...
ENG
IN
9:45 PM
12/1/2024